01

SEGURIDAD

Seguridad & Hardening

Threat model, dependencias, manejo de inputs, CSP y almacenamiento local.

test1 · security-and-hardening

npm audit 0 vulns | 7 fixeos | OWASP en CSV

PROYECTO	FECHA	METODOLOGÍA
inventory-app · Angular 22	Septiembre 2026	agent-skills · OWASP · WCAG 2.1

PENDIENTE lint | PENDIENTE build | PENDIENTE test | PENDIENTE e2e | PENDIENTE auditoría

PENDIENTE npm audit

Test 1 — Seguridad y Hardening

Fecha: 2026-09-10 **Objetivo:** `C:\Users\juanca\Documents\app\inventory-app` (Angular 22, SPA) **Skill aplicado:** `security-and-hardening` + referencia `security-checklist.md` **Verificación:** `npm run lint` ✅ | `npm run build` ✅ | `npm audit` ✅ 0 vulnerabilidades

---

1. Threat Model (resumen)

Fronteras de confianza:

Frontera	Origen de datos	Riesgo
Upload de imágenes	Archivo local del usuario (drag & drop / input)	Archivos maliciosos, payloads SVG
Importación CSV	Archivo local del usuario	Tamaño descontrolado, datos malformados
Exportación CSV	Datos del inventario (nombre, SKU, descripción)	**Fórmulas en Excel/Sheets (CSV injection)**
localStorage	Persistencia del cliente	Lectura no autorizada si el dispositivo se comparte

Assets: datos de inventario (productos, movimientos, proveedores), persistidos sin cifrar en el navegador. Arquitectura: **aplicación 100% cliente (sin backend, sin auth, sin API)**. El modelo de amenazas es local-only; no hay red.

STRIDE por frontera: Spoofing (bajo, sin auth/external), Tampering (formulas CSV), Repudiation (n/a), Information disclosure (localStorage plano), DoS (CSV gigante), Elevation (n/a).

2. Hallazgos

ALTA

- **H1 — CSV formula injection en exportaciones** (`src/app/utils/csv.util.ts`) Los valores de texto que empiezan con `=`, `+`, `-`, `@`, tab o CR se exportan tal cual en `productos-*.csv`, `movimientos-*.csv` y `reposicion-*.csv`. Al abrirse en Excel/Sheets se interpretan como fórmulas (abuso de datos que el propio usuario introduce — SKU/nombre no están restringidos a caracteres al día de hoy).

MEDIA

- **M1 — Validación de imagen solo por MIME** (`src/app/utils/image.util.ts`) `file.type.startsWith('image/')` aceptaba cualquier tipo `image/*` (incl. `image/svg+xml`, que puede llevar payload). Peor: si `canvas.getContext('2d')` devolvía `null`, se resolvía con el **data URL crudo del archivo**, eludiendo la re-codificación a JPEG que hacía de sanitización.

- **M2 — Sin Content-Security-Policy** (`src/index.html`) La app no definía CSP, headers de seguridad ni política de referrer.

BAJA

- **B1 — `JSON.parse(localStorage)` sin validación de forma** (`product/movement/supplier.service.ts`) Se aceptaba cualquier array; un objeto corrupto o malicioso podía romper `list()` / `byId()` o estados derivados. Además, los datos corruptos nunca se limpiaban (quedaban para reactivar el error).
- **B2 — Importación CSV sin límite de tamaño** (`src/app/pages/products/products.component.ts`) `FileReader.readAsText` leía el archivo completo sin cap; un CSV enorme bloqueaba el hilo (DoS local).

INFO

- **I1 — Datos en localStorage sin cifrar.** Arquitectural: aplicación cliente-only. Documentado como limitación, no aplicable como fix (requeriría backend). `crypto.randomUUID()` exige contexto seguro (`localhost`/HTTPS): funcionará en producción solo si se sirve por HTTPS.
- **I2 — `npm audit`: 0 vulnerabilidades** (556 dependencias). Lockfile único, `packageManager: npm@11.19.0`.

3. Fixes aplicados

ID	Archivo(s)	Cambio
H1	`src/app/utils/csv.util.ts`	`escapeCsv` neutraliza celdas de texto que empiezan por `= + - @ \t \r \n` prefijando `'` (OWASP). Los números siguen exportándose sin alterar.
M1	`src/app/utils/image.util.ts`	Allowlist estricta de MIME (`jpeg/png/webp/gif`); si falta `canvas 2d`, se **rechaza** el archivo en vez de devolver el data URL original.
M1	`src/app/components/image-upload/image-upload.component.html`	`accept` restringido a los mismos tipos.
M2	`src/index.html`	CSP meta (`default-src 'self'`, `script-src 'self'`, `style-src 'self' 'unsafe-inline'`, `img-src 'self' data: blob:`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`, `form-action 'self'`) + `referrer=strict-origin-when-cross-origin`.

ID	Archivo(s)	Cambio
B1	`src/app/services/*.service.ts`	Guards `isProduct` / `isMovement` / `isSupplier` (bracket access por `noPropertyAccessFromIndexSignature`): validan la forma antes de aceptar; ante datos corruptos **limpian la clave** y regeneran.
B2	`src/app/pages/products/products.component.ts`	Límite de importación de 5 MB con mensaje de error y reset del input.

4. Verificación

- `npm run lint` → All files pass linting (evidencia: `evidence/lint.txt`)
- `npm run build` → Application bundle generation complete (evidencia: `evidence/build.txt`)
- `npm audit --json` → 0 vulns (evidencia: `evidence/npm-audit.json`)

5. Checklist residual (no aplicable / pendiente)

Ítem	Estado
Auth / sesiones / rate limiting	No aplica (sin backend)
SQL / SSRF / shell	No aplica (sin servidor)
Headers CORS	No aplica (sin API)
Cifrado en reposo (localStorage)	No aplica en cliente-only; documentado (I1)

6. Cierre

Riesgo residual tras el test: bajo para el modelo cliente-only. Ningún hallazgo de alta/moderada queda abierto. Recommended follow-up: migrar a backend con auth si la app se comparte entre usuarios.